

AI-Powered Email Agent

Using Google Agent Development Kit (Google ADK)

TECHNICAL PROJECT REPORT

Submitted to:

Department of IT

Phoenix College of Management and IT

In partial fulfillment of the requirements for the
Extra Course Internal Evaluation

Submitted by:

Bivek Kumar Yadav

B.Tech in Artificial Intelligence

1st Year / 2nd Semester

Student ID:-2530004

GitHub Repository :- https://github.com/bivekkumaryadav622-hub/email_agent_adk

2026

Table of Contents

1. Introduction.....	5
2. Problem Definition.....	5
3. Objectives of the Project.....	6
4. Dataset Description.....	6
5. Tools and Library Used.....	6
6. Methodology.....	7
7. Model Implementation.....	7
8. Results and Evaluation.....	8
9. Discussion.....	8
10. Future Enhancements.....	9
References.....	10
Appendix.....	11

1. Introduction:

- a. Overview of Generative AI: Generative AI systems produce new content or actions from natural-language input. This project connects an LLM to a real tool (`send_email`) so the system performs an actual action rather than only generating text about one.
- b. Role of Python and the LLM: Python defines the ADK agent, the Gmail-authentication logic, and the `send_email` tool, while the LLM (Meta's LLaMA 3.3 70B, accessed via Groq Cloud and LiteLLM) supplies the reasoning that decides when to call that tool.
- c. Project Motivation: Composing and sending routine emails is repetitive. This project explores how a conversational agent can accept a plain-language request, gather the needed details, and safely send the email on the user's behalf, as a hands-on introduction to agentic AI and tool-calling built during the two-week training.

2. Problem Definition:

- a. Description of the problem selected: Manually opening a mail client and filling in the recipient, subject, and body for every email is repetitive. This project reframes that task as something a conversational agent can carry out from an instruction such as “send an email to X about Y.”
- b. Real world relevance: This mirrors the growing use of AI copilots and assistants embedded in productivity tools that draft and send communications on a user's behalf while keeping a human in the loop for approval.
- c. Type of problem: Not a classification/regression/clustering problem – this is an agentic tool-use problem, where an LLM must decide when and how to invoke a real-world action (sending an email) safely.

3. Objectives of the Project:

- a. Learning Goals: Understand how an LLM-based agent decides when to call an external tool; build an agent with Google's Agent Development Kit (ADK); integrate Python with the Gmail API using OAuth 2.0; and apply safe-by-design agent patterns such as requiring explicit user confirmation before any real-world action.
- b. Expected Outcomes: A working agent that gathers email details through conversation, shows a draft, and sends the email only after user confirmation – plus a reusable understanding of the agent-tool-confirmation pattern applicable to other agentic projects.

4. Dataset Description:

- a. Source of Data: Not applicable – this agent does not use a dataset. Its only external inputs at runtime are the recipient address, subject, and body supplied conversationally by the user
- b. Number of records and features: Not applicable – no dataset is used
- c. Data attributes explanation: the three fields the agent works with are the recipient email address (To), the subject line, and the plain-text body, all supplied by the user in conversation and passed directly to the `send_email` tool.

5. Tools and Library Used:

- a. Python Version: Python 3.
- b. Libraries: google-adk (Agent, LiteLlm), litellm, google-api-python-client (Gmail API), google-auth-oauthlib, google-auth, python-dotenv.
- c. IDE / Platform used: Visual Studio Code, with Anaconda for Python environment management.

(Python, Google Agent Development Kit (Google ADK), Gmail API , OAuth 2.0 , REST APIs , Prompt Engineering)

6. Methodology

- a. Data Collection: the recipient, subject, and body are collected conversationally from the user at runtime; there is no upfront dataset collection step.
- b. Data Preprocessing: not applicable in the traditional sense – the agent checks that all three fields (To, Subject, Body) are present, asking follow-up questions for anything missing.
- c. Feature Selection: not applicable – there are no engineered features; the agent works directly from natural-language conversation.
- d. Model Selection: Groq-hosted LLaMA 3.3 70B (Versatile) was selected for its fast inference and strong instruction-following, accessed through LiteLLM inside the ADK agent.
- e. Training and Testing: no model training was performed (a pre-trained LLM is used as-is). Testing consisted of running sample conversations to confirm the agent always drafts, confirms, and only then sends.

7. Model Implementation

- a. Approach Used: an LLM-driven agent (Google ADK Agent) wired to a single custom tool, `send_email`, which authenticates with Gmail via OAuth 2.0 and dispatches the message through the Gmail API
- b. Code Explanation: `_get_gmail_service()` authenticates via a cached or refreshed OAuth token (or a one-time consent flow); `send_email()` builds, base64-encodes, and sends the MIME message through the Gmail API; `root_agent` wires the LLM, instruction, and tool together (*logic only, not full code*)
- c. Hyperparameter tuning: not applicable – no model training/tuning occurs; the only configurable choice is the LLM model itself, left at its default settings (*none performed*).

8. Results and Evaluation

- a. Evaluation approach: since this is not a classifier, success was judged by whether the agent (a) always requested missing fields, (b) always displayed a draft and obtained confirmation, and (c) correctly reported the real Gmail API result.
- b. Sample interaction: when asked to send an email to a colleague about an upcoming meeting, the agent asked for the missing recipient, displayed a draft, and sent the email only after the user confirmed, then reported back the Gmail message ID
- c. Error handling: supplying an invalid recipient address correctly produced an error status with a descriptive detail message instead of a silent failure or a hallucinated success. (Not applicable: no confusion matrix, since this is not a classifier)

9. Discussion

- a. Interpretation of results: a narrowly-scoped, single-tool agent can reliably automate a small but complete real-world task, provided its instruction explicitly enforces a confirm-before-act pattern
- b. Strengths and challenges: the clear separation between language generation (LLM) and the real action (send_email tool), the mandatory confirmation step, and the restricted OAuth scope (gmail.send only) made the agent safe and predictable. The main challenge was writing an instruction precise enough that the LLM never calls the tool before confirmation is given
- c. Summary of Findings: a well-scoped agent with a single, clearly defined tool and an explicit confirmation gate can automate a routine communication task safely and predictably
- d. Skills and Concepts Learned: over the two-week training, skills built included Python/Anaconda setup; accessing LLMs via Groq and building UIs with Streamlit; embeddings and vector databases (Pinecone) and the Gemini API; agent tool-calling fundamentals; MCP and Google ADK concepts; building this Email Sender Agent with Gmail/OAuth integration; and core data science skills (NumPy, Pandas, regression, classification, clustering, and hypothesis testing)

10. Future Enhancements

- a. Extended capabilities: add scopes to read and draft emails (not just send), and integrate a RAG layer so the agent can reference past correspondence or documents when composing emails
- b. Deployment possibilities: deploy the agent behind a simple Streamlit or web chat interface for non-technical users, and combine it with the other tool agents built during training (currency conversion, weather) into a single multi-tool assistant.

References:

The following resources were used in the design and implementation of this project:

[1] Google Agent Development Kit (ADK) Documentation. [Available: google.github.io/adk-docs]

[2] Gmail API Documentation, Google Developers. [Available: <https://developers.google.com/gmail/api>]

[3] Groq Cloud and LiteLLM Documentation. [Available: [console.groq.com](https://console.groq.com/docs), docs.litellm.ai]

[4] Project Repository (Email_Agent_ADK),
GitHub:[https://github.com/bivekkumaryadav622-hub/email_agent_adk].

Appendix:

Project Repository: the complete source code for the Email_Agent_ADK is available at the GitHub link below. Key code excerpt – send_email tool and agent definition – and space for interaction screenshots follow.

```
root_agent = Agent(  
    name="email_agent",
```

```

model=LiteLlm(model="groq/llama-3.3-70b-versatile"),
description="An agent that composes and sends emails on the user's behalf via Gmail.",
instruction=(...),
tools=[send_email],
)

```

Screenshots – Proof of Working Agent;

The screenshot shows the Visual Studio Code interface for a project named EMAIL_ADK_AGENT. The Explorer panel on the left shows the file structure, including agent.py. The Terminal panel on the right shows the output of the command 'adk web'. The output includes a warning about the --reload flag on Windows, followed by information about using in-memory memory service and per-agent artifact storage. It then shows the application startup process, including the creation of an InMemoryCredentialService and the starting of the server process. The final output indicates that the ADK Web Server has started successfully and provides the URL for local testing: http://127.0.0.1:8000.

```

PS C:\VAI & ML\Data Science\EMAIL_ADK_AGENT> a"C
PS C:\VAI & ML\Data Science\EMAIL_ADK_AGENT> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSign
ad) ; (& "c:\VAI & ML\Data Science\EMAIL_ADK_AGENT\venv\Scripts\Activate.ps1")
(venv) PS C:\VAI & ML\Data Science\EMAIL_ADK_AGENT> adk web .
WARNING: The --reload flag is not supported on Windows because it forces Uvicorn to use SelectorEventLoop,
which does not support subprocesses (needed for executing tools). Forcing --no-reload.
2026-07-31 19:41:25,017 - INFO - service_factory.py:266 - Using in-memory memory service
2026-07-31 19:41:25,018 - INFO - local_storage.py:89 - Using per-agent session storage rooted at C:\VAI & ML
\Data Science\EMAIL_ADK_AGENT
2026-07-31 19:41:25,019 - INFO - local_storage.py:121 - Using per-agent artifact storage rooted at C:\VAI & ML
\Data Science\EMAIL_ADK_AGENT
C:\VAI & ML\Data Science\EMAIL_ADK_AGENT\venv\Lib\site-packages\google\adk\cli\fast_api.py:566: UserWarning:
[EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in fut
ure versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
C:\VAI & ML\Data Science\EMAIL_ADK_AGENT\venv\Lib\site-packages\google\adk\auth\credential_service\in_memory
_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental
and may change or be removed in future versions without notice. It may introduce breaking changes at any ti
me.
  super().__init__()
INFO: Started server process [15072]
INFO: Waiting for application startup.

+-----+
| ADK Web Server started |
+-----+
| For local testing, access at http://127.0.0.1:8000. |
+-----+

INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

```

Fig 1: VS Code showing agent.py, with the ADK Dev UI server started successfully in the terminal.

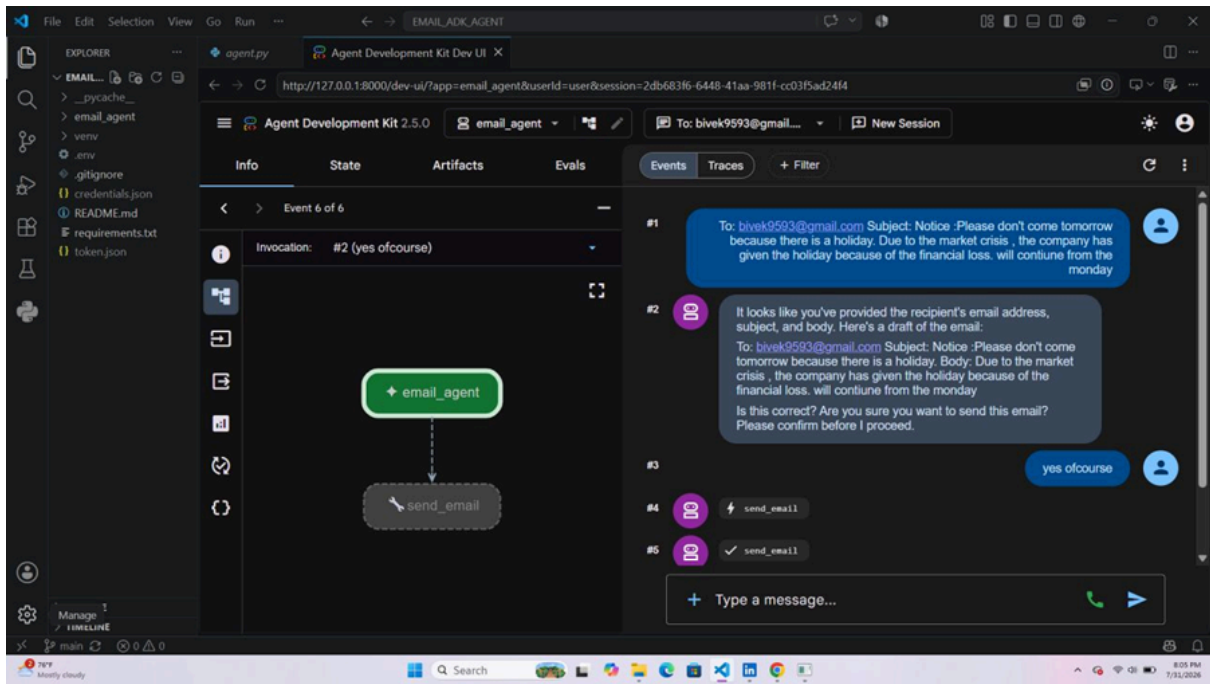


Fig 2: Terminal logs showing the OAuth 2.0 consent flow and the LiteLLM/Groq API calls made by the agent.

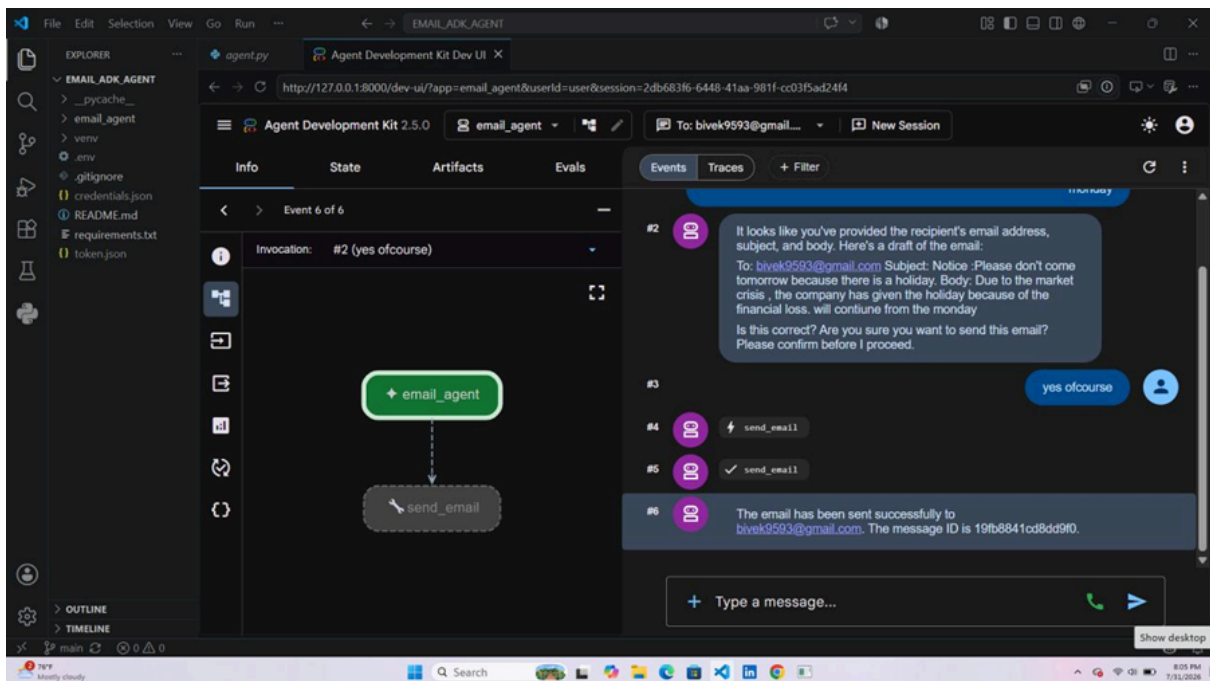


Fig 3: ADK Dev UI – the agent gathers the recipient, subject, and body and prepares a draft (agent → send_email tool graph shown on the left). Then agent revising the draft on request before sending. Again The Agent ask for Confirmation, User confirmation (“ok send it”) triggering the send_email tool call.

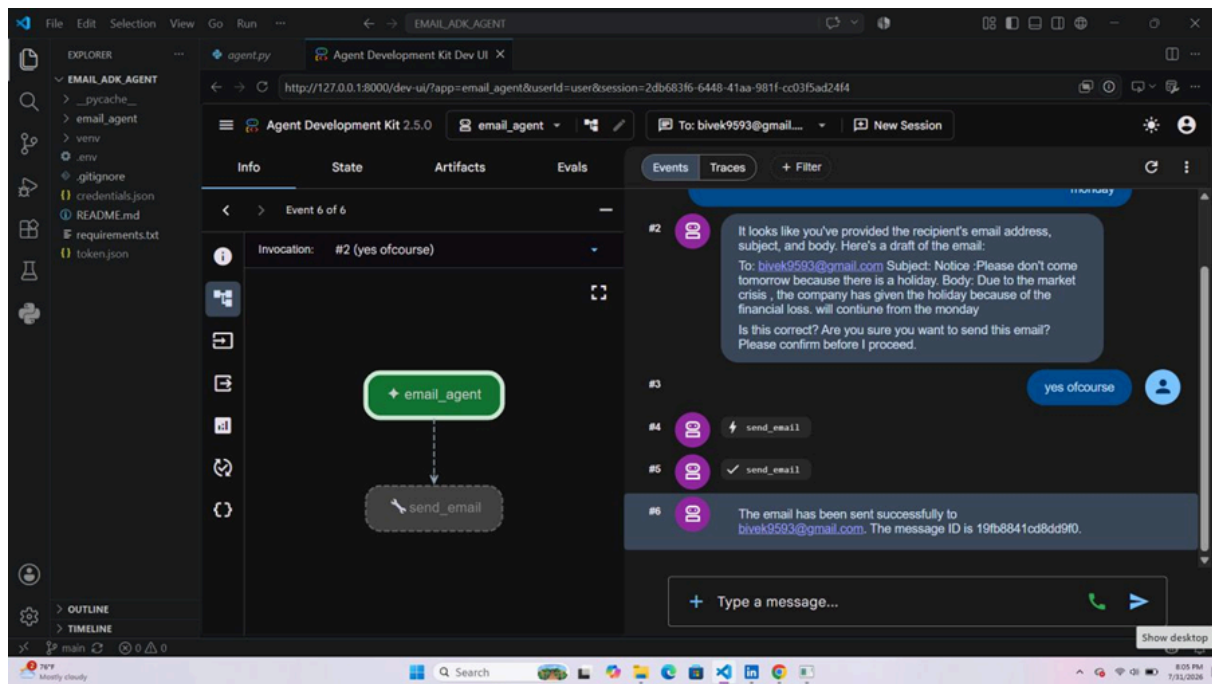


Fig 6: The send_email tool's function response – a successful Gmail API call returning a real message ID.

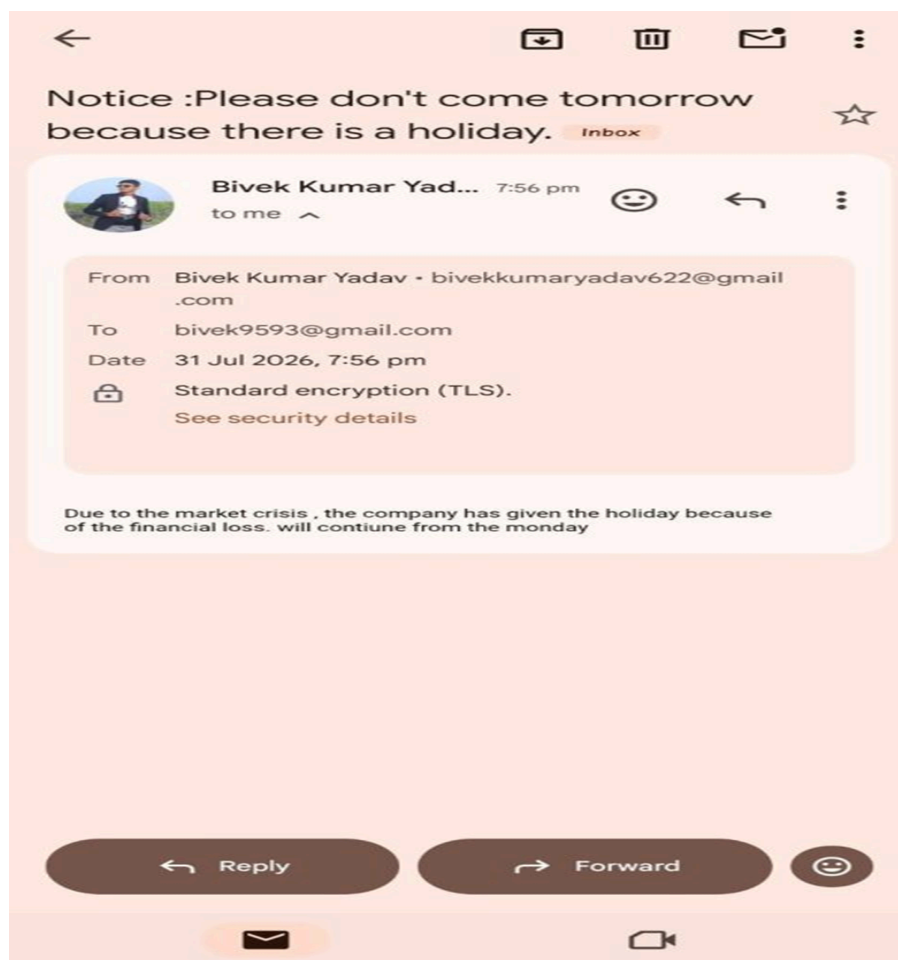


Fig 7: The actual email received in the recipient's Gmail inbox.

AI Powered Email Agent Flowchart

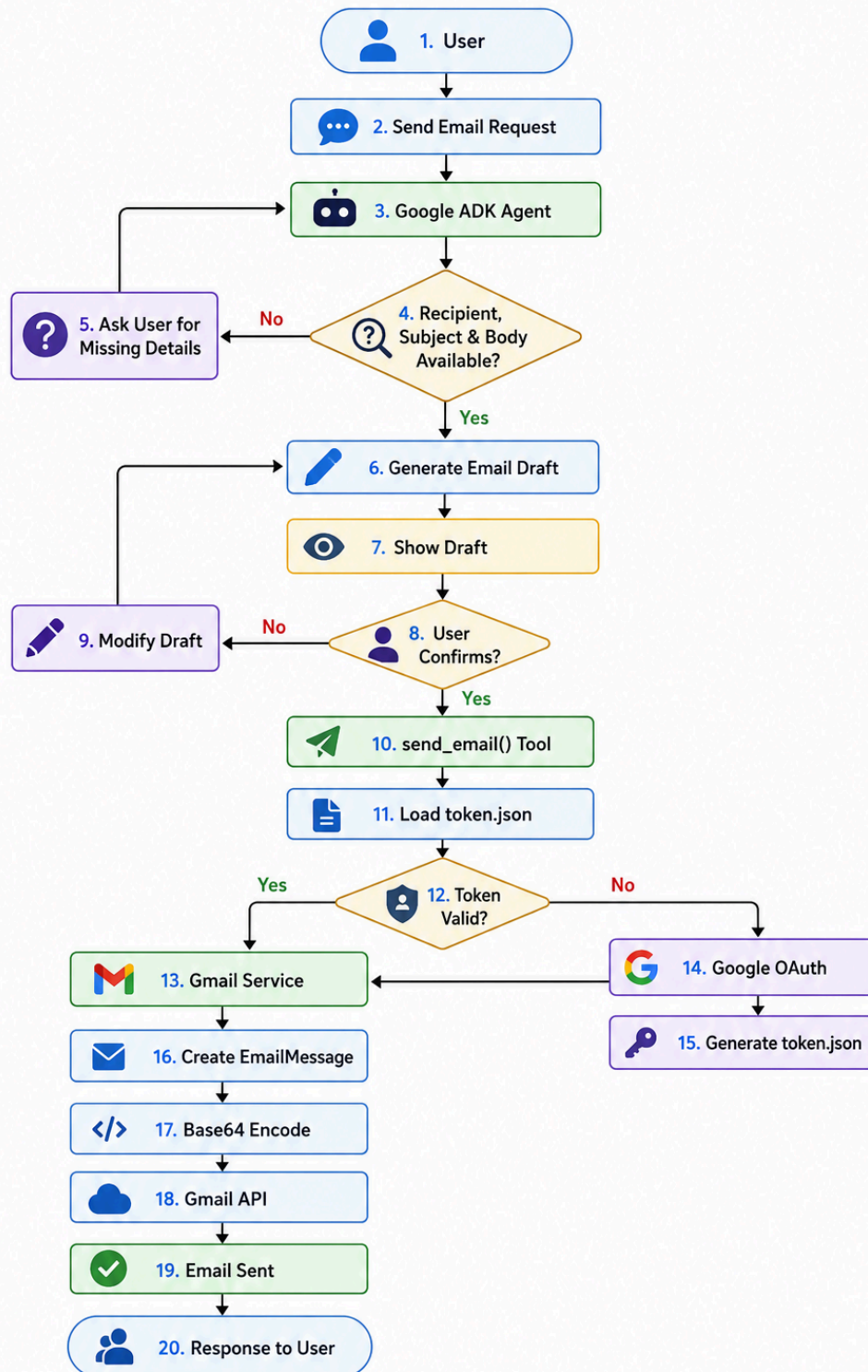


Fig.Source:- [<https://chatgpt.com/c/6a7344c9-ed48-83e8-ac5c-08c0014f9463>]